Bi-Fourier spectral B matrix

Benjamin Ménétrier

Last update: April 14, 2025

Contents

1 General description

This documents describes the `Bifourier` blocks implemented in the SABER library to apply a static background error covariance matrix in spectral space. This code generalizes the Fortran implementation of Berre (2000) in the ALADIN/AROME model.

List of spectral central blocks:

- `BifourierCovariance`: univariate, homogenous and isotropic covariance.
- `BifourierId`: identity (specific for spectral space).

List of spectral outer blocks:

- `BifourierBalance`: balance operator.
- `BifourierCatTPs`: concatenatenation of temperature and logarithm of surface pressure fields.
- `BifourierGridToSpectral`: forward spectral transform.
- `BifourierSpectralToGrid`: inverse spectral transform.
- `BifourierVorDivToRedWind`: conversion from vorticity/divergence to reduced wind.
- `BifourierVorToPb`: addition of balanced pressure, computed from vorticity (and the backward operation).

List of non-spectral outer blocks:

- `Biperiodization`: biperiodization of 2D fields.
- `RedWindToGeoWind`: conversion from reduced wind to geographical wind.

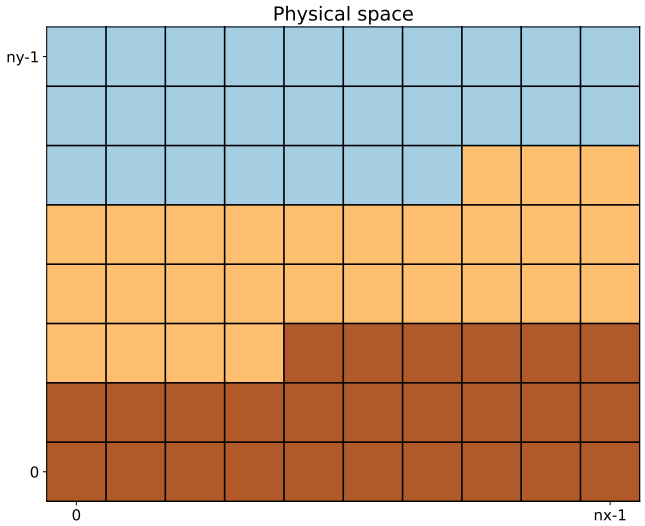
List of auxillary classes:

- `BifourierAromeLegacy`: reading and processing of AROME files (in binary or NetCDF format), to use their data in the `BifourierBalance` and `BifourierCovariance` blocks.
- `BifourierTransform`: spectral transform, parallelization, derivatives, etc.
- `BifourierTransformStore`: storage of `BifourierTransform` objects to avoid redundant setups.
- `BiperiodizationImpl`: biperiodization implementation.

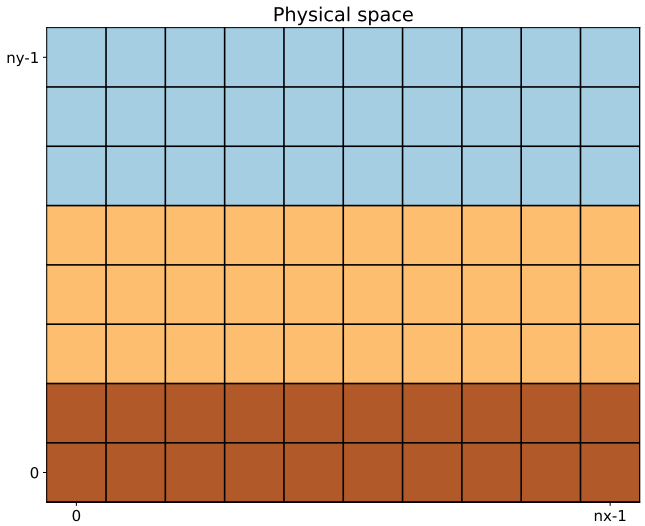
2 Spectral transform

2.1 Parallelization

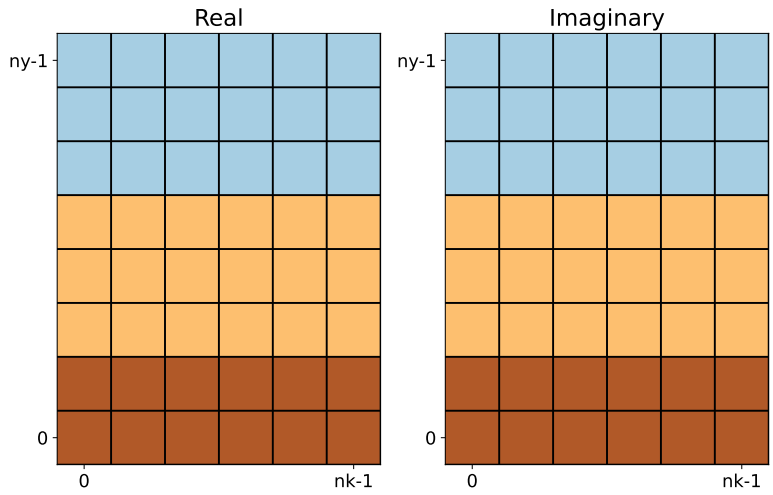
The spectral transform process starts with a field of size $n_x \times n_y$ in physical space, split among several MPI tasks. For instance:



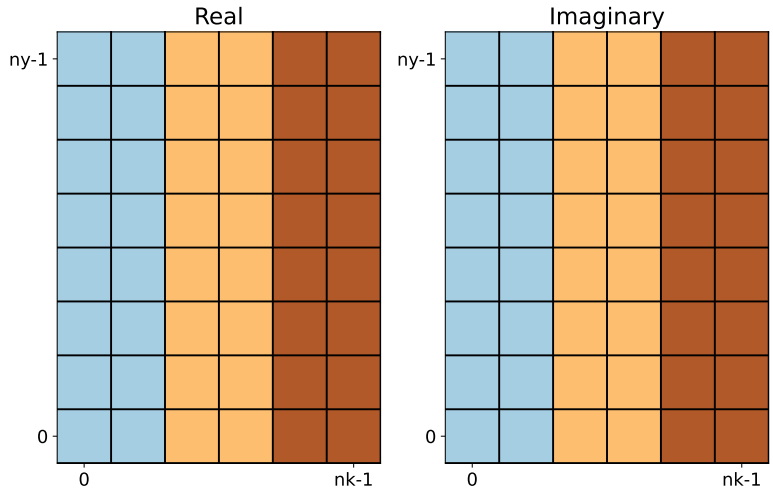
The first step is a global communication (`MPI_Alltoallv`) to make sure that each row is handled by a single task:



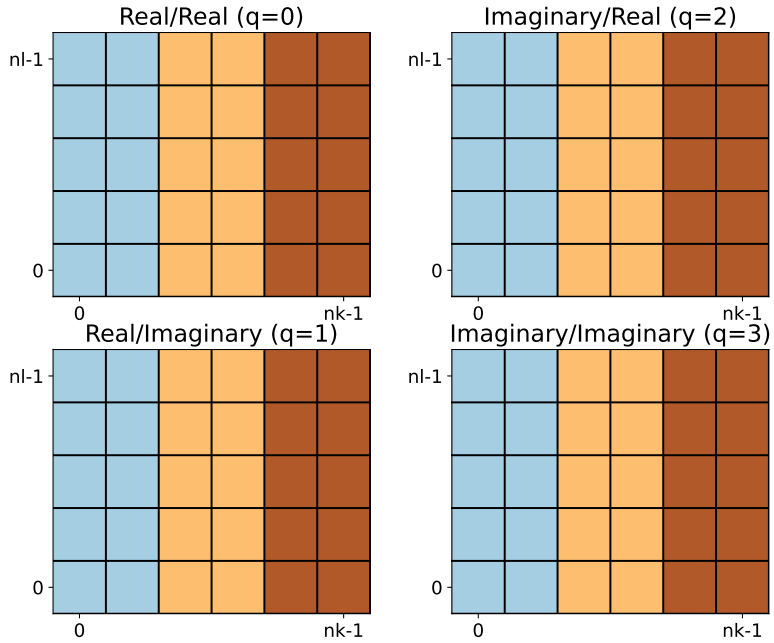
The second step is a FFT along each row, leading to two arrays (real and imaginary components) of size $n_k \times n_y$:



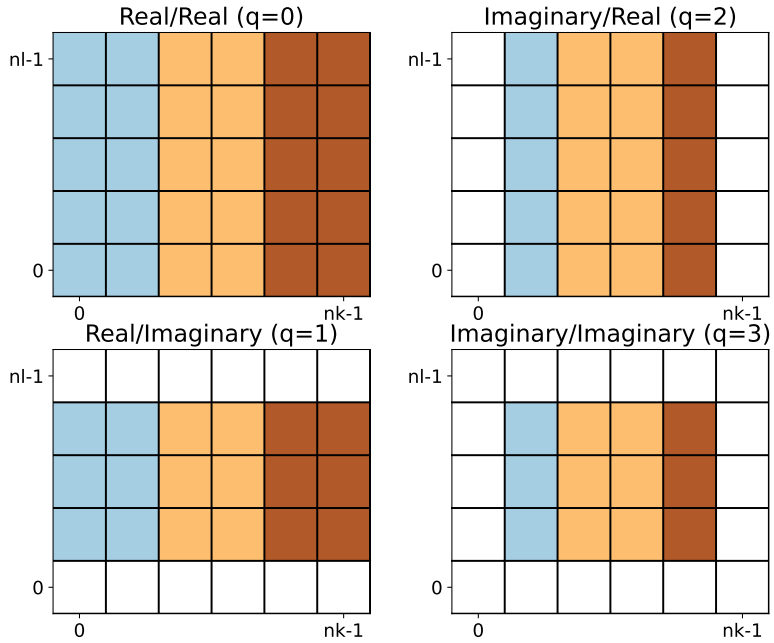
The third step is a global communication (`MPI_Alltoallv`) to make sure that each column is handled by a single task:



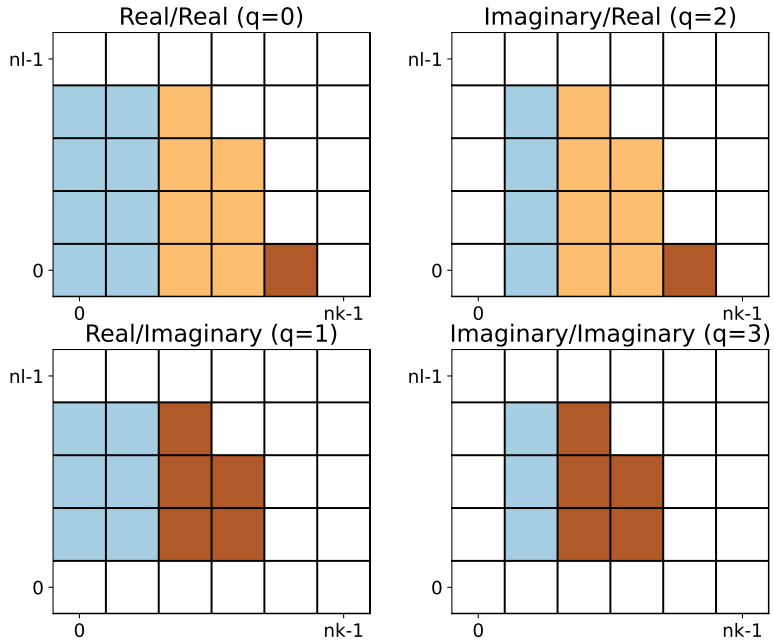
The fourth step is a FFT along each column, leading to four arrays (real/real, real/imaginary, imaginary/real and imaginary/imaginary components) of size $n_k \times n_l$:



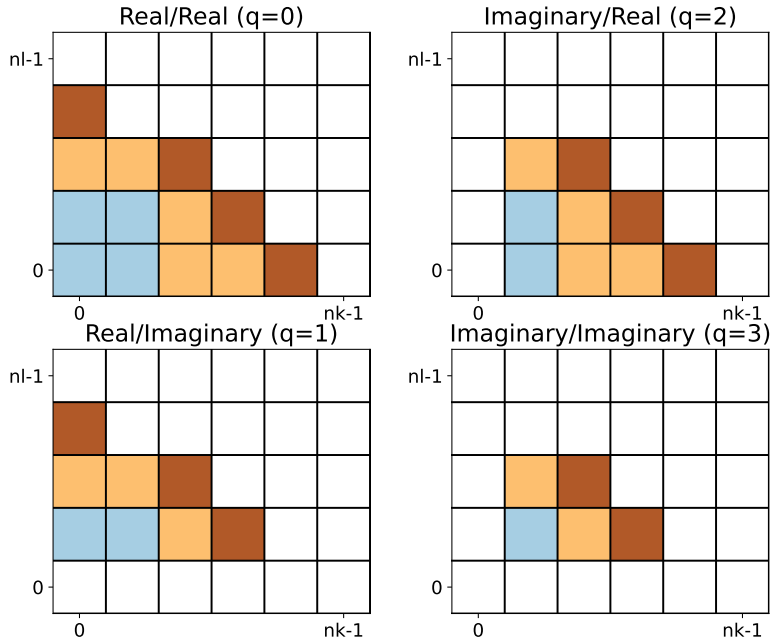
The fifth step is the removal of redundant spectral coefficients because the input field is real. It is easy to check that the number of remaining spectral cofficients is precisely equal to the number of input grid points in physical space (same number of degrees of freedom):



The sixth step is an elliptic truncation on spectral wavenumbers to ensure isotropy:



The seventh step is a global communication (`MPI_Alltoallv`) to split the remaining wavenumbers into equal chunks and improve the load balancing during calculations. It is important to keep all components of each couple (k, l) on the same task to compute derivatives:



Hereafter, we call "forward process" the sequence of these seven steps, and "inverse process" the backward sequence.

2.2 Normalization

The spectral transforms based on FFTW are not normalized, so a the spectral coefficient must be multiplied by $\sqrt{n_x n_y}$ at the end of the forward process, and at the beginning of the inverse process.

2.3 Parseval's identity

The Parseval's identity states that the canonical norm (i.e. the squared canonical inner product of a field with itself) is the same in physical and spectral spaces. To verify the Parseval's identity with the decomposition described above, is is necessary to multiply each spectral coefficients at the end of the forward process by the number of remaining components after the fifth step for its own (k,l). For instance here:

- spectral coefficient for $(k,l)=(0,0)$ is multiplied by 1,
- spectral coefficients for $(k,l)=(1,0)$ is multiplied by 2,
- spectral coefficients for $(k,l)=(0,1)$ is multiplied by 2,
- spectral coefficients for $(k,l)=(1,1)$ is multiplied by 4,
- etc.

Of course, the input spectral coefficients must be divided by the same amount at the beginning of the inverse process. This operation has one major advantage: the adjoint of the forward process is now equivalent to the inverse process.

2.4 Total wavenumber

To ensure isotropy, all the B matrix operators are based on the total wavenumber $k^\star$:

$$k^\star(k,l)=\max(n_{_k}-1,n_{_l}-1)\sqrt{\frac{k^2}{(n_{_k}-2)^2}+\frac{l^2}{(n_{_l}-2)^2}} \tag{1}$$

3 Vorticity / divergence to wind

3.1 Derivatives

In spectral space, computing derivatives is straightforward. Assuming that each spectral coefficient is defined by the triplet (k,l,q):

- Derivative in zonal direction: $\mathbf{s}_{\text{out}}=\mathbf{D}_{x}\mathbf{s}_{\text{in}}$ with

$$\mathbf{s}_{\text{out}}(k,l,0)=\frac{2\pi k}{L_{_x}}\,\mathbf{s}_{\text{in}}(k,l,2) \tag{2a}$$

$$\mathbf{s}_{\text{out}}(k,l,1)=\frac{2\pi k}{L_{_x}}\,\mathbf{s}_{\text{in}}(k,l,3) \tag{2b}$$

$$\mathbf{s}_{\text{out}}(k,l,2)=-\frac{2\pi k}{L_{_x}}\,\mathbf{s}_{\text{in}}(k,l,0) \tag{2c}$$

$$\mathbf{s}_{\text{out}}(k,l,3)=-\frac{2\pi k}{L_{_x}}\,\mathbf{s}_{\text{in}}(k,l,1) \tag{2d}$$

  where $L_{_x}$ is the domain width in the zonal direction.

- Derivative in meridional direction: $\mathbf{s}_{\text{out}} = \mathbf{D}_y \mathbf{s}_{\text{in}}$ with

$$\mathbf{s}_{\text{out}}(k,l,0) = \frac{2\pi l}{L_y}\, \mathbf{s}_{\text{in}}(k,l,1) \tag{3a}$$

$$\mathbf{s}_{\text{out}}(k,l,1) = -\frac{2\pi l}{L_y}\, \mathbf{s}_{\text{in}}(k,l,0) \tag{3b}$$

$$\mathbf{s}_{\text{out}}(k,l,2) = \frac{2\pi l}{L_y}\, \mathbf{s}_{\text{in}}(k,l,3) \tag{3c}$$

$$\mathbf{s}_{\text{out}}(k,l,3) = -\frac{2\pi l}{L_y}\, \mathbf{s}_{\text{in}}(k,l,2) \tag{3d}$$

  where L_y is the domain width in the meridional direction.

In both directions, $\mathbf{s}_{\text{out}}(k,l,q)$ is set to zero if the corresponding $\mathbf{s}_{\text{in}}$ component has been removed at step five of the forward process.

3.2 Laplacian

The Laplacian operator and its inverse are even simpler to compute:

- Forward Laplacian: $\mathbf{s}_{\text{out}} = \mathbf{L}\mathbf{s}_{\text{in}}$ with

$$\mathbf{s}_{\text{out}}(k,l,q) = -\left(\frac{4\pi^2 k^2}{L_x^2} + \frac{4\pi^2 l^2}{L_y^2}\right)\, \mathbf{s}_{\text{in}}(k,l,q) \tag{4}$$

- Inverse Laplacian: $\mathbf{s}_{\text{out}} = \mathbf{L}^{-1}\mathbf{s}_{\text{in}}$ with

$$\mathbf{s}_{\text{out}}(k,l,q) = -\frac{1}{\dfrac{4\pi^2 k^2}{L_x^2} + \dfrac{4\pi^2 l^2}{L_y^2}}\, \mathbf{s}_{\text{in}}(k,l,q) \tag{5}$$

  and $\mathbf{s}_{\text{out}}(0,0,0) = 0$

3.3 Wind variables

We denote:

- λ: longitude
- φ: latitude
- u: eastward wind
- v: northward wind
- u_g: geographical wind in x direction

- v_g: geographical wind in y direction
- u_r: reduced wind in x direction
- v_r: reduced wind in y direction
- ψ: stream function
- χ: velocity potential
- ζ: vorticity
- δ: divergence

The spherical wind (u, v) are related to the geographical wind (u_g, v_g) through the grid projection Jacobian:

$$u = \alpha_\lambda \left(\frac{\partial x}{\partial \lambda} u_g + \frac{\partial y}{\partial \lambda} v_g \right) \tag{6a}$$

$$v = \alpha_\varphi \left(\frac{\partial x}{\partial \varphi} u_g + \frac{\partial y}{\partial \varphi} v_g \right) \tag{6b}$$

where α_λ and α_φ are normalization coefficients:

$$\alpha_\lambda = \left(\left(\frac{\partial x}{\partial \lambda} \right)^2 + \left(\frac{\partial y}{\partial \lambda} \right)^2 \right)^{-1/2} \tag{7a}$$

$$\alpha_\varphi = \left(\left(\frac{\partial x}{\partial \varphi} \right)^2 + \left(\frac{\partial y}{\partial \varphi} \right)^2 \right)^{-1/2} \tag{7b}$$

$$\tag{7c}$$

The reduced wind is equal to the geographical wind divided by the local map factor m:

$$u_r = \frac{u_g}{m} \tag{8a}$$

$$v_r = \frac{v_g}{m} \tag{8b}$$

Computing (ζ, δ) from reduced wind is then straightforward:

$$\zeta = \mathbf{D}_x v_r - \mathbf{D}_y u_r \tag{9a}$$

$$\delta = \mathbf{D}_x u_r + \mathbf{D}_y v_r \tag{9b}$$

The inverse operations requires two steps:

$$\psi = \mathbf{L}^{-1} \zeta \tag{10a}$$

$$\chi = \mathbf{L}^{-1} \delta \tag{10b}$$

and

$$u_r = \mathbf{D}_x \chi - \mathbf{D}_y \psi \tag{11a}$$

$$v_r = \mathbf{D}_x \psi + \mathbf{D}_y \chi \tag{11b}$$

4 Balance and covariance operators

A general description of balance operators is available here: `https://github.com/benjaminmenetrier/vertical_balance/blob/main/vertical_balance.pdf`

In the spectral framework, balance operators are designed with a vertical regression matrix for each total wavenumber $k^\star$. While the `BifourierBalance` block code is generic enough to handle any balance configuration, the standard one Berre (2000) is given by:

$$\begin{pmatrix} P_b \\ \delta \\ (T, P_s) \\ q \end{pmatrix} = \begin{pmatrix} \mathbf{I} & 0 & 0 & 0 \\ \mathbf{M} & \mathbf{I} & 0 & 0 \\ \mathbf{N} & \mathbf{P} & \mathbf{I} & 0 \\ \mathbf{Q} & \mathbf{R} & \mathbf{S} & \mathbf{I} \end{pmatrix} \begin{pmatrix} P_b \\ \delta_u \\ (T, P_s)_u \\ q_u \end{pmatrix} \tag{12}$$

with:

- P_b: balanced pressure, defined by $P_b = \mathbf{H}\zeta$
- ζ: vorticity
- δ: divergence
- (T, P_s): concatenation of temperature and logarithm of surface pressure at the bottom
- q: specific humidity

The subscript u denotes the unbalanced variables.

The univariate covariance operator is designed with a vertical covariance matrix for each total wavenumber $k^\star$, but its actually its square-root that is stored and applied (adjoint and forward successively).

References

Berre L. 2000. Estimation of synoptic and mesoscale forecast error covariances in a limited-area model. *Monthly Weather Review* **128**(3): 644–667.